NOTE Modeling events is a fascinating topic that goes beyond Spring and would require a few chapters to cover properly. If you'd like to learn more about this subject, I recommend reading these articles by Martin Fowler: "Focusing on Events" (https://martinfowler.com/eaaDev/EventNarrative.xhtml); "Domain Event" (https://martinfowler.com/eaaDev/DomainEvent.xhtml); and "What do you mean by 'Event-Driven'?" (https://martinfowler.com/articles/ 201701-event-driven.xhtml), all on his MartinFowler.com blog.

The output of the function can be the simple identifier of the packed order represented as a `Long` object.

Now that input and output are clear, it's time to define the function. Create a new `DispatchingFunctions` class, and add a `pack()` method to implement order packing as a function.

Listing 10.5 Implementing the "pack" action as a function

```
package com.polarbookshop.dispatcherservice;

import java.util.function.Function;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class DispatchingFunctions {
  private static final Logger log =
    LoggerFactory.getLogger(DispatchingFunctions.class);

  public Function<OrderAcceptedMessage, Long> pack() {     ❶
    return orderAcceptedMessage -> {                       ❷
      log.info("The order with id {} is packed.",
        orderAcceptedMessage.orderId());
      return orderAcceptedMessage.orderId();               ❸
    };
  }
}
```

❶ Function implementing the order-packing business logic

❷ It takes an OrderAcceptedMessage object as input.

❸ Returns an order identifier (Long) as output

You can see how there's only standard Java code in this listing. I strive to provide real-world examples in this book, so you might wonder what's